OVERCLOCK: MODELO DIDÁCTICO INTERACTIVO PARA ENSEÑAR SISTEMAS OPERATIVOS (PLANIFICACIÓN DE PROCESOS)

ANTEPROYECTO

INTRODUCCIÓN

Los sistemas operativos constituyen una herramienta fundamental en el uso cotidiano de los dispositivos informáticos, ya que todo software desarrollado se ejecuta sobre alguno de estos. Por esta razón, comprender sus conceptos y funcionamiento resulta esencial para cualquier profesional en el área de la computación, ya que permite optimizar de manera eficiente los procesos involucrados en el desarrollo y ejecución de aplicaciones.

PROBLEMA

El estudio de los sistemas operativos abarca una amplia variedad de temas que, por su complejidad, pueden representar un desafío para los estudiantes. Sin embargo, no existen herramientas didácticas que faciliten su comprensión y aprendizaje.

OBJETIVOS

Objetivo general:

Desarrollar un modelo de software educativo didáctico he interactivo con el propósito de facilitar la comprensión de los sistemas operativos, enfocado principalmente en el tema de planificación de procesos.

Objetivos particulares:

- Investigar los conceptos fundamentales de los sistemas operativos que serán aplicados en el proyecto.
- Analizar el funcionamiento básico del motor gráfico Godot Engine y su lenguaje GDScript orientado al desarrollo 2D.
- Diseñar mecánicas que refuercen el carácter educativo del proyecto en el área de sistemas operativos.
- Implementar las mecánicas previamente definidas conforme a la planificación.
- Corregir errores detectados durante el proceso de desarrollo del software.
- Evaluar la efectividad del videojuego mediante pruebas con usuarios para medir su impacto educativo.

METODOLOGÍA

El desarrollo del proyecto seguirá un enfoque iterativo, lo que permitirá la realización de mejoras constantes a lo largo del proceso.

- Se llevará a cabo una investigación basada en fuentes académicas sobre sistemas operativos, Godot Engine, GDScript y planificación de procesos.
- Se planificarán las mecánicas del software mediante el uso de diagramas de flujo.
- Se desarrollarán los componentes y scripts necesarios para la construcción del software educativo.
- Se realizarán pruebas interinas e iterativas orientadas a la detección y corrección de errores.
- Se aplicará una encuesta diagnóstica a un grupo piloto de estudiantes antes y después de la utilización del software.
- Se evaluará la efectividad del proyecto mediante la comparación de las respuestas obtenidas en ambos momentos.

HERRAMIENTAS

- Motor gráfico: Godot Engine 4.4.
- Diseño gráfico: MediaBang y Procreate.
- Control de versiones: GitHub.
- Lenguaje de programación: GDScript.

ALCANCES Y LIMITACIONES

- El software estará diseñado exclusivamente para computadoras que operen con Windows 11.
- El contenido educativo solo incluirá la enseñanza de la planificación de procesos y la gestión básica de memoria.
- El sistema simulado será de tipo monousuario y monohilo.

JUSTIFICACIÓN

El software podrá constituir una herramienta complementaria a los métodos tradicionales de enseñanza, como presentaciones y ejercicios, los cuales pueden resultar abstractos y no siempre logran mantener la atención de los estudiantes.

MARCO TEÓRICO

SISTEMA OPERATIVO

El sistema operativo (SO) es el software fundamental encargado de gestionar el hardware, el resto del software y los recursos de la computadora. Su función principal es coordinar los servicios esenciales del sistema y proporcionar una interfaz que facilite la interacción del usuario con el hardware y los programas instalados. La presencia de un sistema operativo es indispensable, ya que sin él no sería posible utilizar una computadora de manera funcional.

PLANIFICACIÓN DE PROCESOS

Es una función fundamental de los **sistemas operativos** que consiste en decidir **qué proceso** debe ejecutarse en un momento dado en la CPU.

Cuando hay múltiples procesos listos para ejecutarse, el sistema operativo utiliza algoritmos de planificación para **asignar el tiempo de CPU** de manera eficiente, justa y, en muchos casos, buscando optimizar el rendimiento general del sistema.

ALGORITMOS DE PLANIFICACIÓN DE PROCESOS

FIFO:

La planificación FIFO ejecuta los procesos en el orden en que llegan; el primer proceso que entra en la cola de listos es el primero en ser asignado a la CPU.

SPN:

La planificación SPN selecciona para su ejecución el proceso que tenga el menor tiempo estimado de ejecución, favoreciendo los trabajos más cortos para reducir el tiempo promedio de espera.

Highest Response Ratio Next:

La planificación HRRN se basa en el tiempo de espera y el tiempo estimado de ejecución del proceso, escogiendo al que tenga mayor Ratio de respuesta ya que eso indica que el proceso ha estado esperando más tiempo en relación con el tiempo que requiere para ejecutarse.

$$\text{Proceso} = \frac{(\text{tiempo de espera} + \text{tiempo de servicio})}{\text{tiempo de servicio}}$$

GODOT ENGINE 4.4 Y GDSCRIPT

Godot Engine 4.4:

Es un motor de desarrollo basado en un sistema de nodos, donde cada nodo posee características específicas. El motor sigue el paradigma de programación orientada a objetos, permitiendo que los nodos hijos hereden las propiedades y comportamientos de sus nodos padre.

GDScript:

GDScript es el lenguaje de programación nativo de Godot, diseñado con una sintaxis similar a Python. Es un lenguaje tipado que admite programación orientada a objetos y dispone de un sistema de señales que permite la comunicación entre scripts; cuando un script emite una señal, todos los scripts suscritos a esta son notificados.

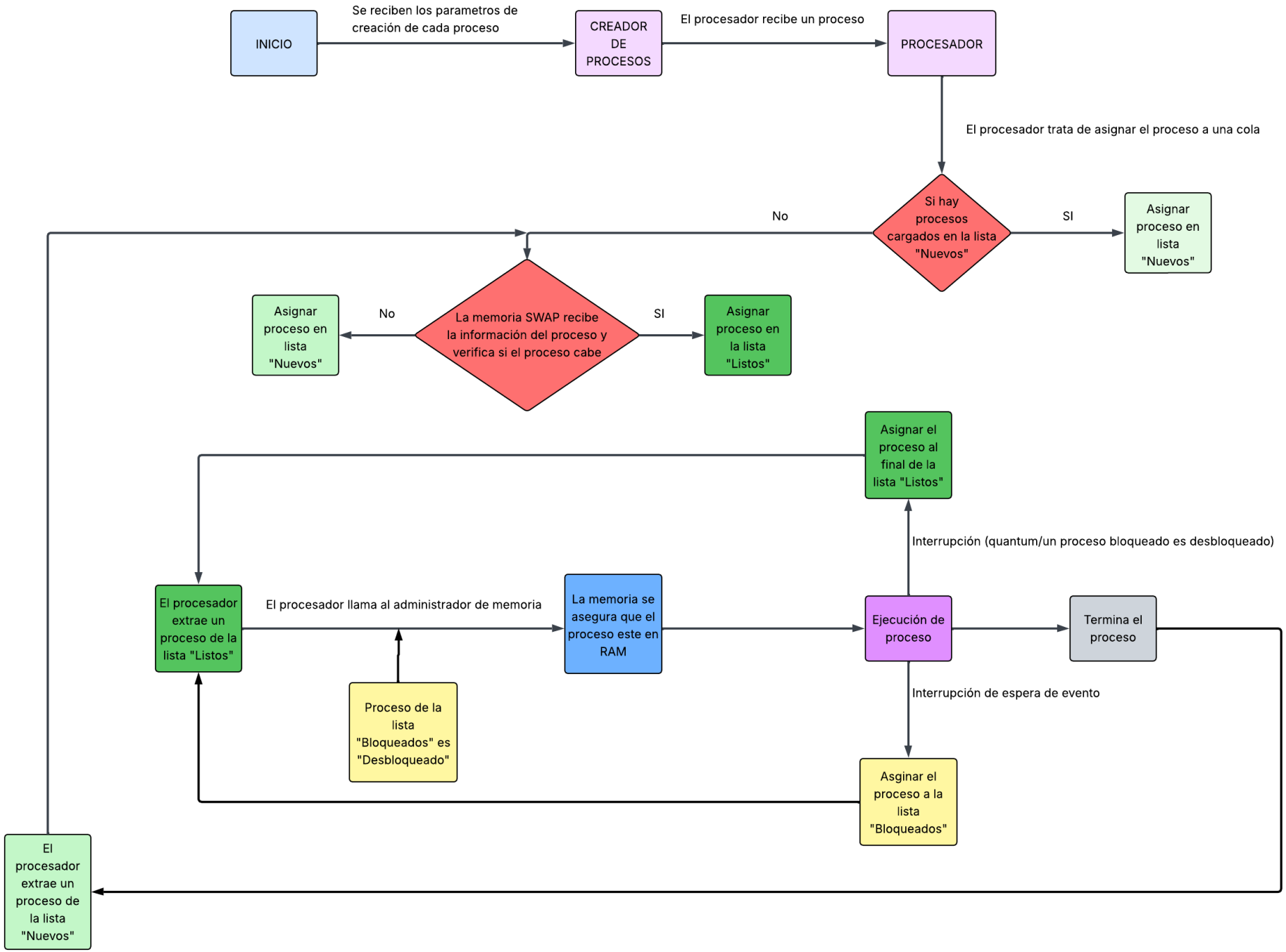
DISEÑO DEL PROYECTO

DISEÑO DE LAS MECÁNICAS PRINCIPALES

Inicialmente se realizaron planificaciones para el desarrollo de los componentes clave del sistema operativo, tomando como base un diagrama de flujo que representa el ciclo de vida de un proceso. A partir de este, se implementaron los componentes principales del videojuego, entre los cuales se encuentran:

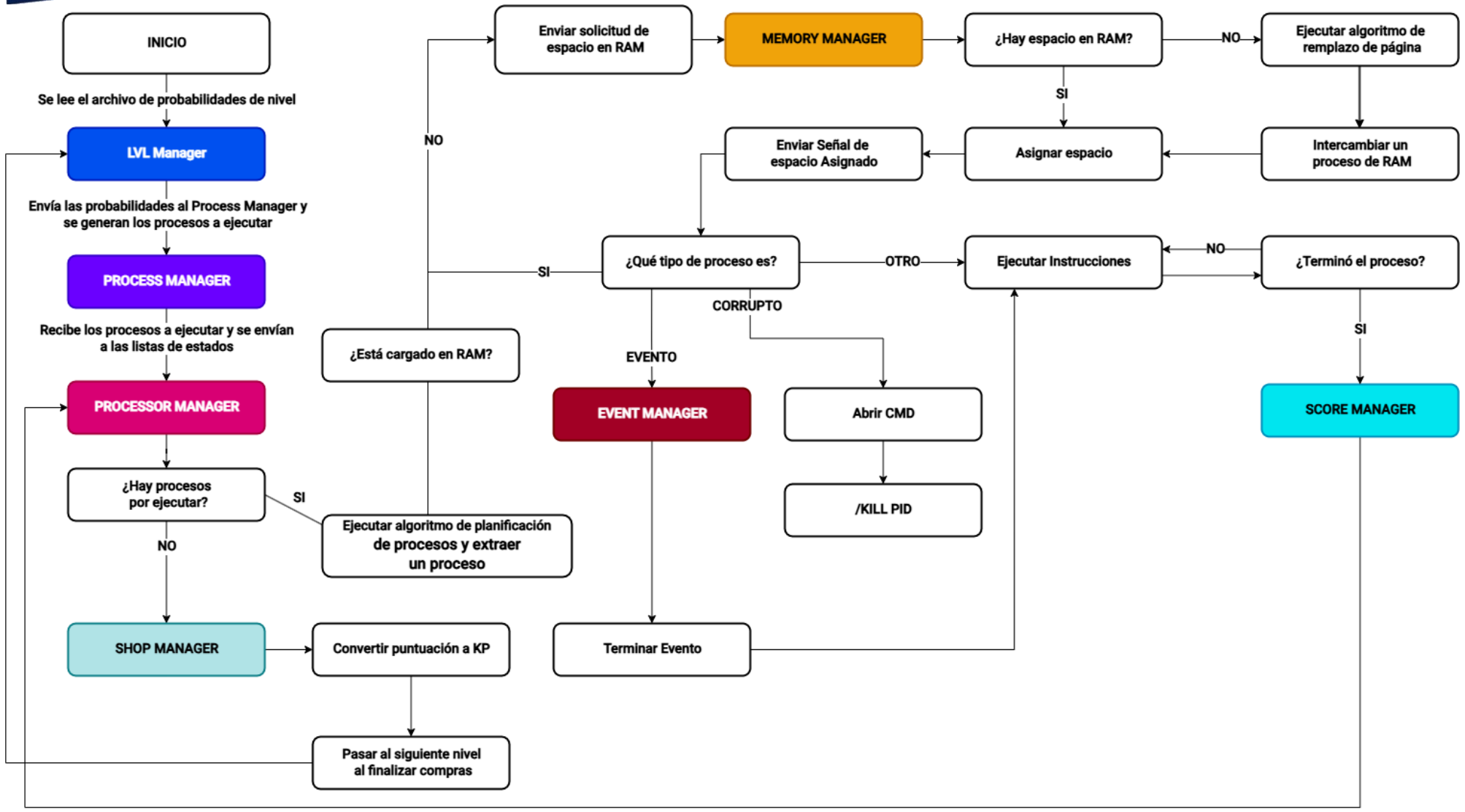
- **Creador de procesos**: genera los procesos necesarios para cada nivel, de acuerdo con los parámetros definidos.
- **Procesador**: gestiona la ejecución de los procesos.
- **Administrador de memoria**: simulan la asignación y administración de la memoria.




INICIO
Se reciben los parametros de creación de cada proceso
CREADOR DE PROCESOS
El procesador recibe un proceso
PROCESADOR
El procesador trata de asignar el proceso a una cola
Si hay procesos cargados en la lista "Nuevos"
SI
Asignar proceso en lista "Nuevos"
No
La memoria SWAP recibe la información del proceso y verifica si el proceso cabe
SI
Asignar proceso en la lista "Listos"
No
Asignar proceso en lista "Nuevos"
Asignar el proceso al final de la lista "Listos"
Interrupción (quantum/un proceso bloqueado es desbloqueado)
El procesador extrae un proceso de la lista "Listos"
El procesador llama al administrador de memoria
La memoria se asegura que el proceso este en RAM
Ejecución de proceso
Termina el proceso
Proceso de la lista "Bloqueados" es "Desbloqueado"
Interrupción de espera de evento
Asginar el proceso a la lista "Bloqueados"
El procesador extrae un proceso de la lista "Nuevos"

DISEÑO DE LAS MECÁNICAS PRINCIPALES DEL VIDEOJUEGO

Una vez definidas las mecánicas principales del sistema se procedió a planificar las mecánicas principales del videojuego mediante un diagrama de flujo para posteriormente desarrollarlas


INICIO
Se lee el archivo de probabilidades de nivel
LVL Manager
Envía las probabilidades al Process Manager y se generan los procesos a ejecutar
PROCESS MANAGER
Recibe los procesos a ejecutar y se envían a las listas de estados
PROCESSOR MANAGER
¿Hay procesos por ejecutar?
SI
NO
Ejecutar algoritmo de planificación de procesos y extraer un proceso
¿Está cargado en RAM?
NO
Enviar solicitud de espacio en RAM
MEMORY MANAGER
¿Hay espacio en RAM?
NO
Ejecutar algoritmo de remplazo de página
SI
Intercambiar un proceso de RAM
Asignar espacio
Enviar Señal de espacio Asignado
¿Qué tipo de proceso es?
SI
OTRO
EVENTO
CORRUPTO
EVENT MANAGER
Abrir CMD
/KILL PID
Terminar Evento
Ejecutar Instrucciones
NO
¿Terminó el proceso?
SI
SCORE MANAGER
SHOP MANAGER
Convertir puntuación a KP
Pasar al siguiente nivel al finalizar compras

SISTEMA DE NIVELES

Se implementó un archivo CSV para cargar la información correspondiente a cada nivel del videojuego. Esta arquitectura permite diseñar y modificar niveles de forma rápida y precisa, facilitando la escalabilidad del proyecto.
El archivo contiene información clave como:

- El número del nivel.
- La cantidad de procesos que aparecen en dicho nivel.
- La presencia o no de un jefe especial.
- La probabilidad de aparición de cada tipo de proceso por nivel.
- El rango de instrucciones que puede tener cada tipo de proceso, según el nivel.

	column	column 2	column 3	column 4	column 5	column 6	column 7	column 8	column 9	column 10	column	column 12	column 13	column	column	column	column	colum
1	lvl	long process probability	normal process probability	corrupt process probability	short process probability	event process probability	long min size	long max size	normal min size	normal max size	corrupt min size	corrupt max size	short min size	short max size	event min size	event max size	number of processes	boss
2	-1	0.0	0.0	0.0	0.0	1	10	16	16	6	12	4	3	2	12	5	100	0
3	1	0.0	0.0	0.0	0.8	0.2	1	12	16	6	12	4	6	2	4	5	4	0
4	2	0.0	0.2	0.1	0.5	0.2	12	24	6	12	4	6	2	4	2	5	6	0
5	3	0.05	0.2	0.1	0.45	0.2	14	28	6	14	5	7	2	5	3	5	8	0
6	4	0.1	0.2	0.15	0.35	0.2	16	30	8	14	5	8	3	5	1	5	10	0
7	5	0.0	0.0	0.0	0.95	0.05	20	40	10	20	6	12	1	2	5	5	64	1
8	6	0.2	0.2	0.15	0.25	0.2	20	34	10	16	6	10	3	6	3	5	12	0
9	7	0.25	0.2	0.15	0.2	0.2	20	36	10	18	6	10	3	6	3	5	14	0
10	8	0.3	0.2	0.2	0.1	0.2	22	38	10	18	6	12	4	6	3	5	16	0
11	9	0.35	0.2	0.2	0.05	0.2	24	40	12	20	6	12	4	7	3	5	18	0
12	10	0.1	0.1	0.6	0.0	0.2	28	44	12	20	10	20	4	6	3	5	25	1
13	11	0.4	0.2	0.15	0.05	0.2	26	42	12	20	7	14	4	8	3	5	20	0
14	12	0.45	0.2	0.2	0.03	0.2	28	44	12	22	7	14	4	8	3	5	22	0
15	13	0.5	0.2	0.2	0.0	0.2	30	46	14	22	8	14	5	8	3	5	24	0
16	14	0.55	0.15	0.25	0.0	0.2	32	48	14	24	8	15	5	9	3	5	26	0
17	15	0.0444	0.7111	0.0	0.0444	0.2	34	50	50	100	10	16	6	10	3	5	20	1
18	16	0.6	0.0857	0.1714	0.0	0.2	36	52	16	24	8	16	6	10	3	5	30	0
19	17	0.6429	0.0857	0.1286	0.0	0.2	38	54	16	26	9	17	6	10	3	5	32	0
20	18	0.6857	0.0857	0.0857	0.0	0.2	40	56	18	26	9	18	6	10	3	5	34	0
21	19	0.7286	0.0429	0.0857	0.0	0.2	42	58	18	28	10	18	6	10	3	5	36	0
22	20	0.8	0.0	0.0	0.0	0.2	80	120	20	30	10	20	6	10	3	5	50	1

CREACIÓN DE PROCESOS

```
88  func _on_timer_timeout() -> void:
89      if numberProcess <= 0:
90          timer.stop()
91          return
92      ProcessType  = ProcessTypeSelector()
93      ProcessRangeSize = ProcessSizeSelector()
94      create_new_process(ProcessType, ProcessRangeSize)
95      IncreasePID()
96      numberProcess -= 1
97
98
99  func StartLvl() -> void:
100     timer = Timer.new()
101     timer.wait_time = 0.5
102     timer.one_shot = false
103     timer.autostart = true
104     timer.connect("timeout", Callable(self, "_on_timer_timeout"))
105     add_child(timer)
106
107 func ConfigLvl(Data : Variant) -> void:
108     #despues cargamos las propiedades del nivel
109     probabilities =  ExtractProbabilities(Data)
110     sizes = ExtractSizes(Data)
111     numberProcess = extractNumberProcess(Data)
112     PrintLog.info("Numero de procesos del nivel: %d", [numberProcess])
113     SendNumberProcess.emit(numberProcess)
114     StartLvl()
115
116
117 func _on_lvl_controller_send_lvl_data(Data:Variant, lvl:Variant) -> void:
118     localLvl = lvl
119     pid = 1
120     ConfigLvl(Data)
```

PROCESADOR

```
extends Node2D
#listas de procesos
var newsProcesses_Queue : Array
var readyProcesses_Queue : Array #suspendidos
var blockedProcesses_Queue : Array #procesos bloqueados por un evento
var finishedProcesses_Queue : Array
#--------------
```

PROCESADOR

```
func _process(delta: float) -> void:
    if ProcessorDelay(delta) == 0:

        if newsProcesses_Queue and AnswerSwapController == false and ProcessEnd == true:
            longTermPlanner(null, 150)#para planificar
            ProcessEnd = false
        if blockedProcesses_Queue and AnswerEventController == false:
            EventProcessPlanner(null, 150)

        Dispatcher()
```

ALGORITMOS DE PLANIFICACIÓN

```gdscript
class_name ProcessSchedules extends Node


func FIFO(Ready_queue : Array) -> Process:
    var CurrentProcess = Ready_queue[0]
    return CurrentProcess

func SPN(Ready_queue : Array) -> Process:
    var CurrentProcess = Ready_queue[0]
    for process in Ready_queue:
        if process.instructions < CurrentProcess.instructions:
            CurrentProcess = process
    return CurrentProcess

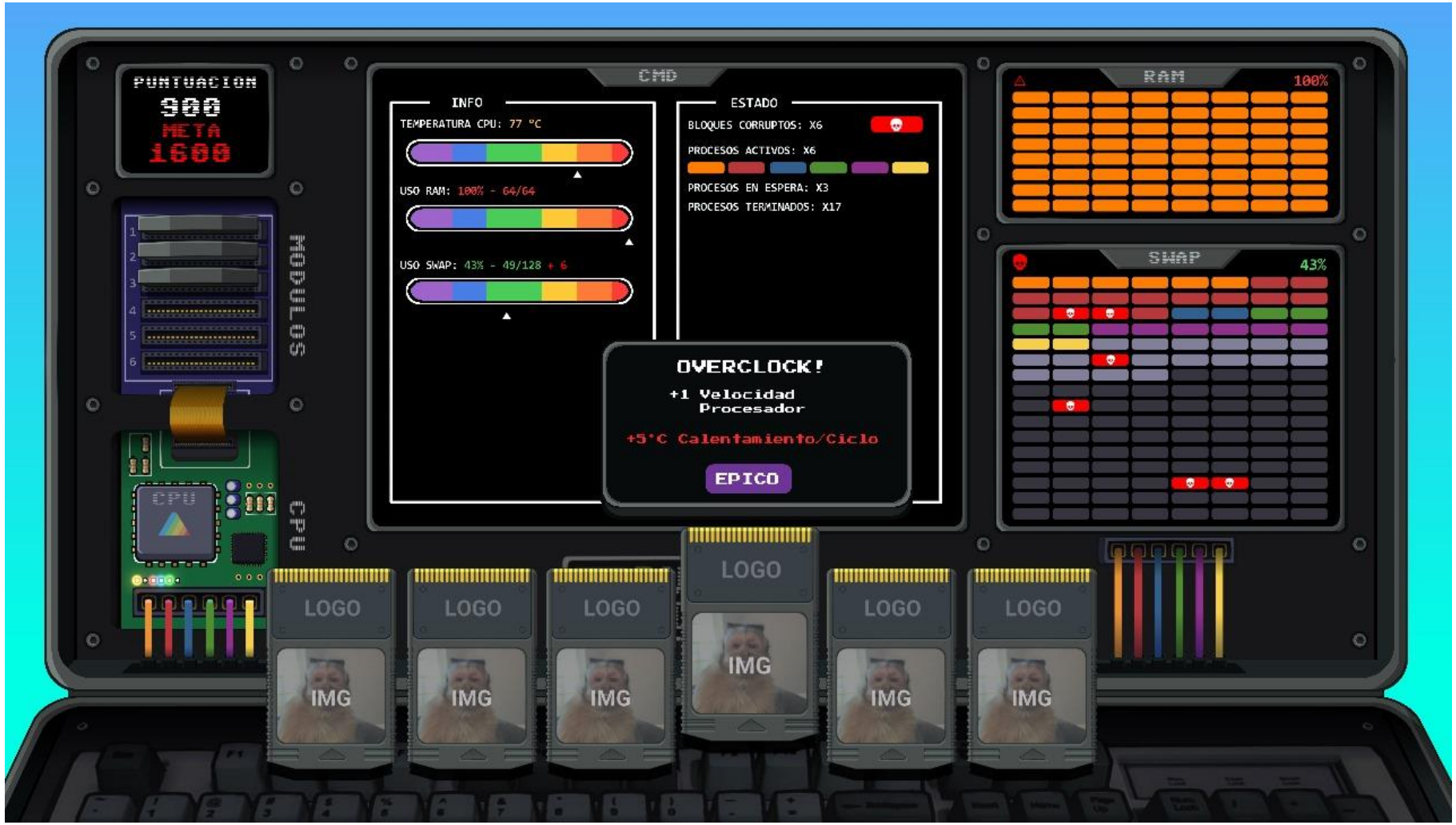
func HRRN(Ready_queue : Array) -> Process:
    var CurrentProcess = Ready_queue[0]

    for process in Ready_queue:
        var WaitTimeProcess = (Time.get_ticks_msec() / 1000.0) - process.TimeCreation
        var ServiceTimeProcess = process.instructions #
        var RatioProcess = (WaitTimeProcess + ServiceTimeProcess) / ServiceTimeProcess

        var WaitTimeCurrentProcess = (Time.get_ticks_msec() / 1000.0) - CurrentProcess.TimeCreation
        var ServiceTimeCurrentProcess = CurrentProcess.instructions #
        var RatioCurrentProcess = (WaitTimeCurrentProcess + ServiceTimeCurrentProcess) / ServiceTimeCurrentProcess

        if RatioProcess > RatioCurrentProcess:
            CurrentProcess = process
    return CurrentProcess
```

PROTOTIPO DE LA INTERFAZ



INTERFAZ FINAL



ANÁLISIS DE RESULTADOS

RETROALIMENTACIÓN DE USUARIO

- La interfaz resultó intuitiva y sencilla para los usuarios.
- El seguimiento del flujo del sistema fue sencillo.
- Hubo buena comprensión de los conceptos de sistemas operativos.
- Motivación para seguir aprendiendo.

CONCLUSIÓN

Este proyecto tiene el potencial de motivar a los estudiantes y mejorar su comprensión de los conceptos relacionados con sistemas operativos, explorando así nuevas formas de aprendizaje más dinámicas e interactivas.

Como trabajo futuro, se propone ampliar el contenido del videojuego para incluir más conceptos de sistemas operativos, tales como la **gestión de disco** o el **manejo de procesadores multihilo**.